

Arquitectura — CertManager

0. Diagramas (contexto y flujo)

Diagrama 0 — Contexto

CertManager visto como una única caja, con sus **actores** y los **sistemas externos** con los que se integra. El nivel de detalle interno (NGINX, Gunicorn, Scheduler) está en el Diagrama 1.

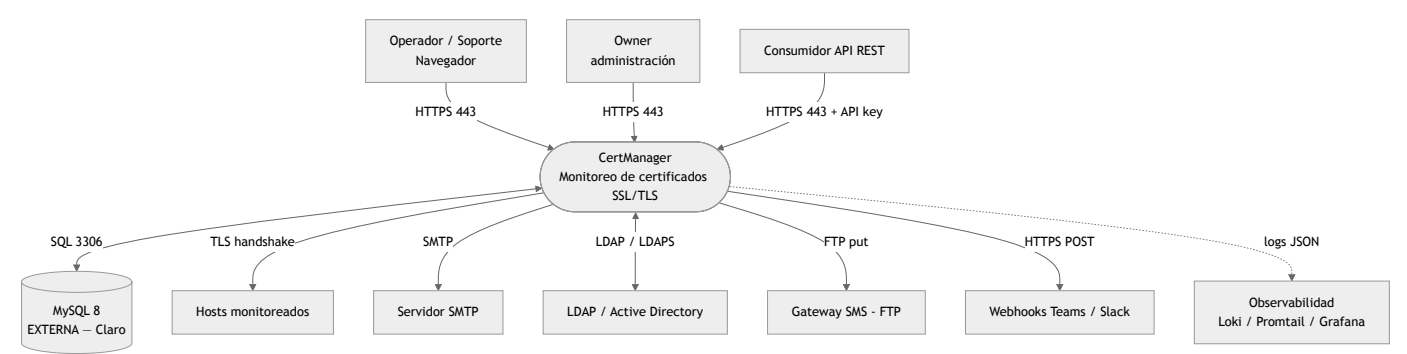
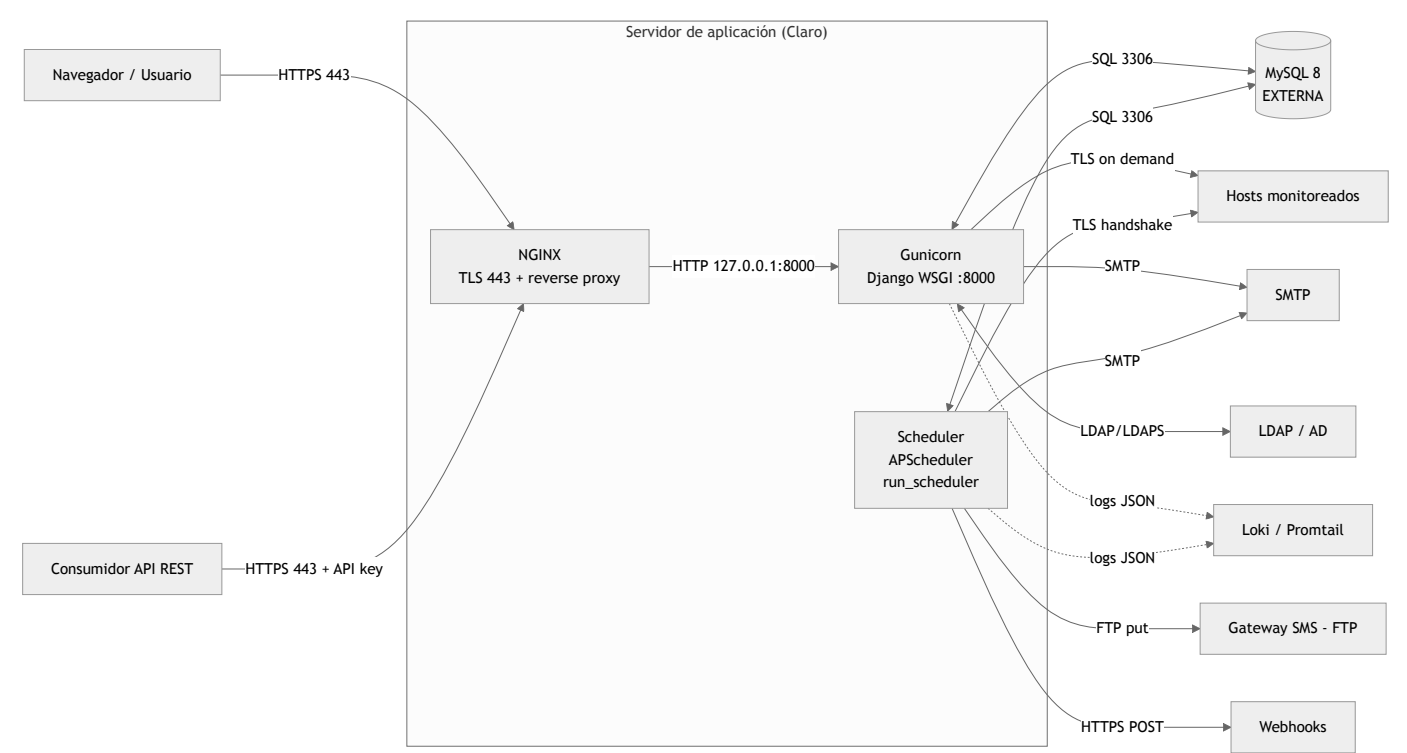


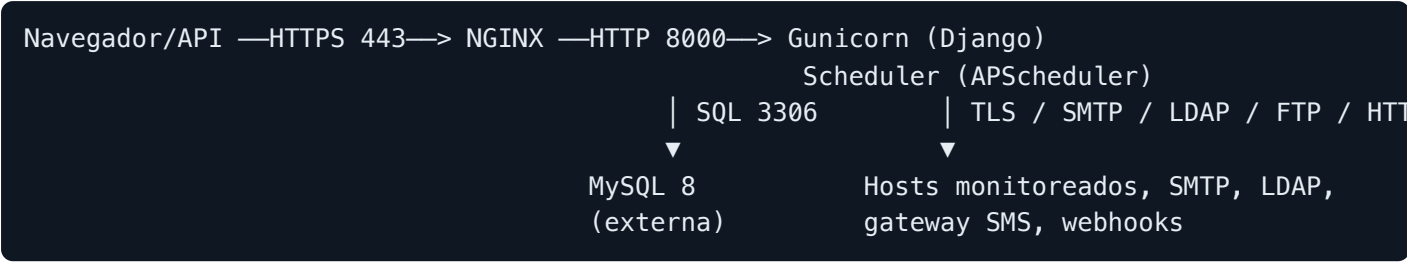
Diagrama 1 — Flujo (nivel contenedor)

Las piezas internas del servidor de aplicación y el flujo de datos entre ellas. El detalle por flujo (puerto, sentido, disparo) está en `01_diagrama_flujo_datos.md`.



1. Visión general

Aplicación monolítica Django (UI server-rendered + API REST) con un planificador en-proceso. Despliegue de 3 piezas: **NGINX** (TLS 443) → **Gunicorn** (app) + **Scheduler** (tareas), contra una **MySQL externa**.
Detalle del flujo de datos en [01_diagrama_flujo_datos.md](#).



2. Componentes (módulos)

Módulo (<code>apps/</code>)	Responsabilidad
<code>core</code>	Configuración global (OrganizationSettings), API keys, auditoría (AuditLog), middlewares (gate de admin, 2FA, expiración/timeout de sesión, log de request), backup.
<code>accounts</code>	Usuarios (login por email), Owner global, 2FA TOTP, lockout, validador de contraseña.
<code>teams</code>	Grupos (Team) + membresías (VIEWER/CONTRIBUTOR/ADMIN).
<code>certificates</code>	Certificados (dominio, puerto, ubicación, grupos), destinatarios, historial de chequeos.
<code>monitoring</code>	SSLChecker , orquestador <code>run_check</code> , scheduler, comandos (<code>check_certificates</code> , <code>data_update_certs_app</code>).
<code>alerts</code>	Alertas, entregas (AlertDelivery), webhooks, notificadores (correo/webhook/SMS).
<code>mailtemplates</code>	Plantillas de correo (builder por bloques).
<code>reports</code>	Reportes programados (CSV/PDF/Excel/correo).
<code>web</code>	Vistas Forge UI (dashboard, certs, grupos, usuarios, configuración, perfil).
<code>api</code>	Router DRF (<code>/api/certificates</code> , <code>/teams</code> , <code>/alerts</code>).

3. Perfiles de configuración (`config/settings/`)

Perfil	Uso	BD
<code>base</code>	común	—
<code>local</code>	desarrollo	SQLite
<code>standalone</code>	pruebas / VDI	SQLite (auto SECRET_KEY)
<code>prod</code>	producción	MySQL 8 (vars <code>DB_*</code>), HTTPS forzado

4. Integraciones externas

Integración	Cómo	Configurable en
Hosts monitoreados	Handshake TLS (núcleo)	Por certificado

Integración	Cómo	Configurable en
SMTP	Envío de alertas/reportes	Panel Correo
LDAP/AD	Verificación de credenciales (login transparente)	Panel LDAP
SMS	Depósito por FTP en gateway corporativo	Panel Integraciones
Webhooks	POST HTTPS (Teams/Slack)	Panel Integraciones
Observabilidad	Logs JSON a stdout → Loki (Promtail); opcional <code>obsforge</code>	Entorno

5. Persistencia y estado

- **MySQL** (externa): única fuente de verdad (certs, usuarios, alertas, config, auditoría).
- **Estáticos**: servidos por la app (WhiteNoise), sin CDN.
- **Sin estado en memoria** compartido entre procesos (el scheduler usa un *lock* por archivo para ser singleton).

6. Seguridad (resumen)

HTTPS forzado (HSTS, cookies `Secure`), CSP, X-Frame DENY, anti-SSRF en chequeos y webhooks, RBAC por grupo, API keys hasheadas, secretos por entorno. Detalle en `09_seguridad_y_cumplimiento.md`.

7. Decisiones de arquitectura (rationale)

- **Monolito + scheduler en-proceso**: simplicidad operativa; sin colas/servicios externos.
- **MySQL externa**: estándar corporativo; la app no gestiona la BD.
- **Server-rendered + HTMX**: sin SPA/build pesado; menos superficie de ataque (estáticos locales).
- **3 modos de despliegue** (Linux/Docker/K8s) sobre el mismo artefacto.